

k-Vizinhos mais Próximos

k-Vizinhos Mais Próximos

- ◆ *Aprendizado baseado em instâncias* simplesmente armazena os exemplos de treinamento.
- ◆ Quando um novo exemplo (caso) precisa ser classificado, esse exemplo é comparado com os exemplos armazenados.
- ◆ A classificação é decidida a partir da similaridade entre o exemplo a ser classificado e os exemplos armazenados.

k-Vizinhos Mais Próximos

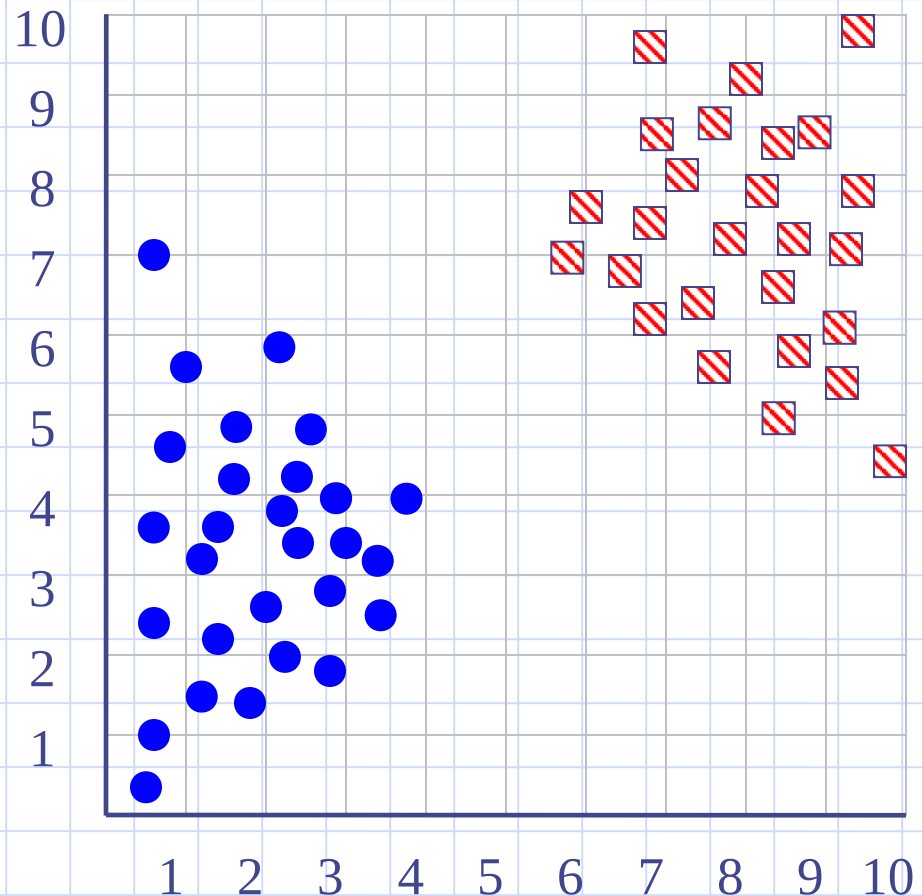
- ◆ Aprendizado baseado em instâncias é chamado também de aprendizado “*lazy*”.
- ◆ Isso se deve ao fato do processamento ser atrasado até o momento de classificação de um novo exemplo.

k-Vizinhos Mais Próximos

- ◆ Dois métodos bastante conhecidos de aprendizado baseado em instâncias são:
 - k-vizinhos mais próximos;
 - Regressão com pesos locais.
- ◆ A idéia do método k-vizinhos mais próximos é bastante simples...

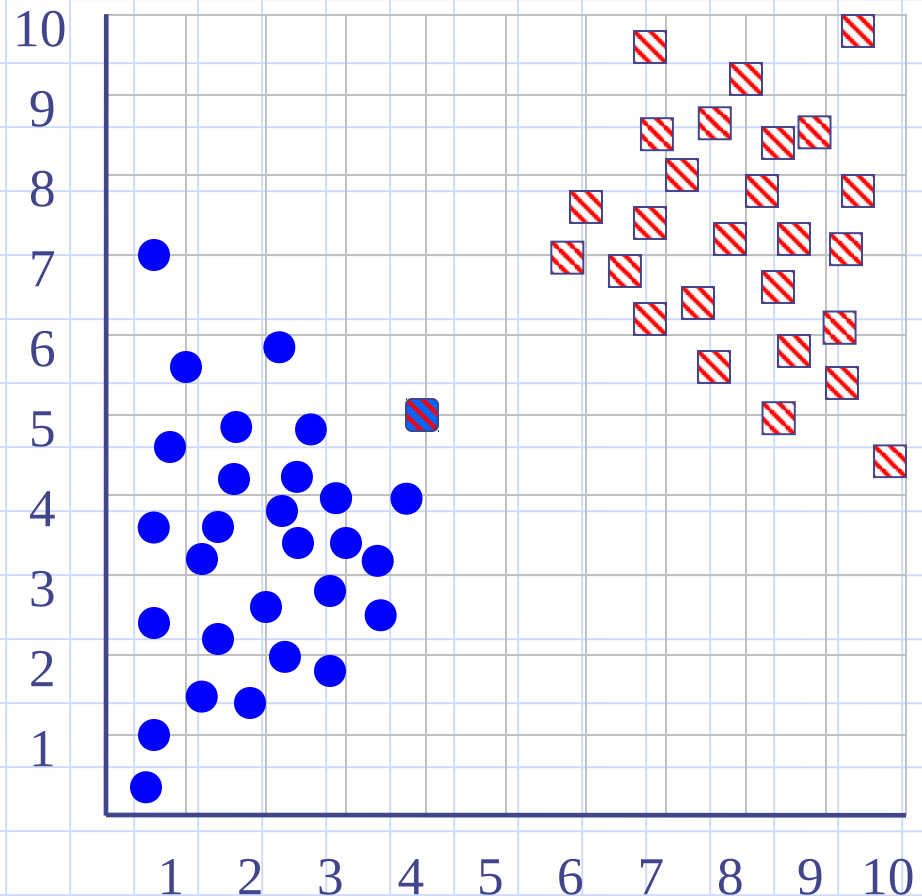
K-Vizinhos mais Próximos

◆ Inicialmente, os exemplos de treinamento são armazenados.



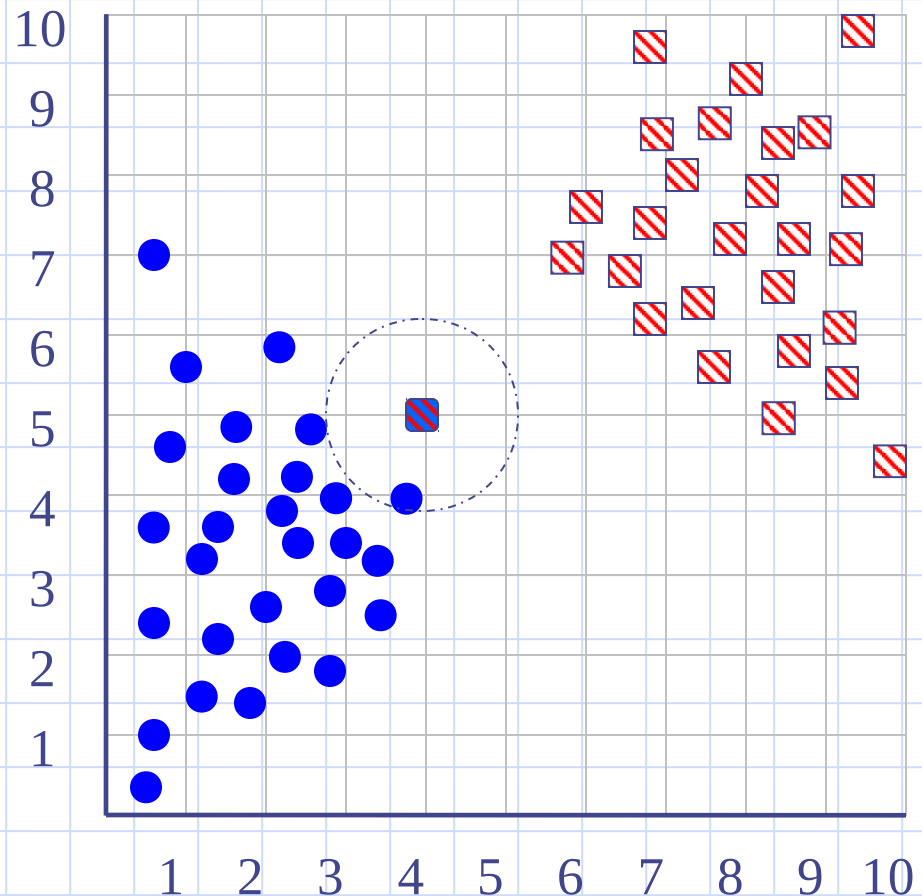
K-Vizinhos mais Próximos

- ◆ Inicialmente, os exemplos de treinamento são armazenados.
- ◆ Quando uma novo exemplo precisa ser classificado...



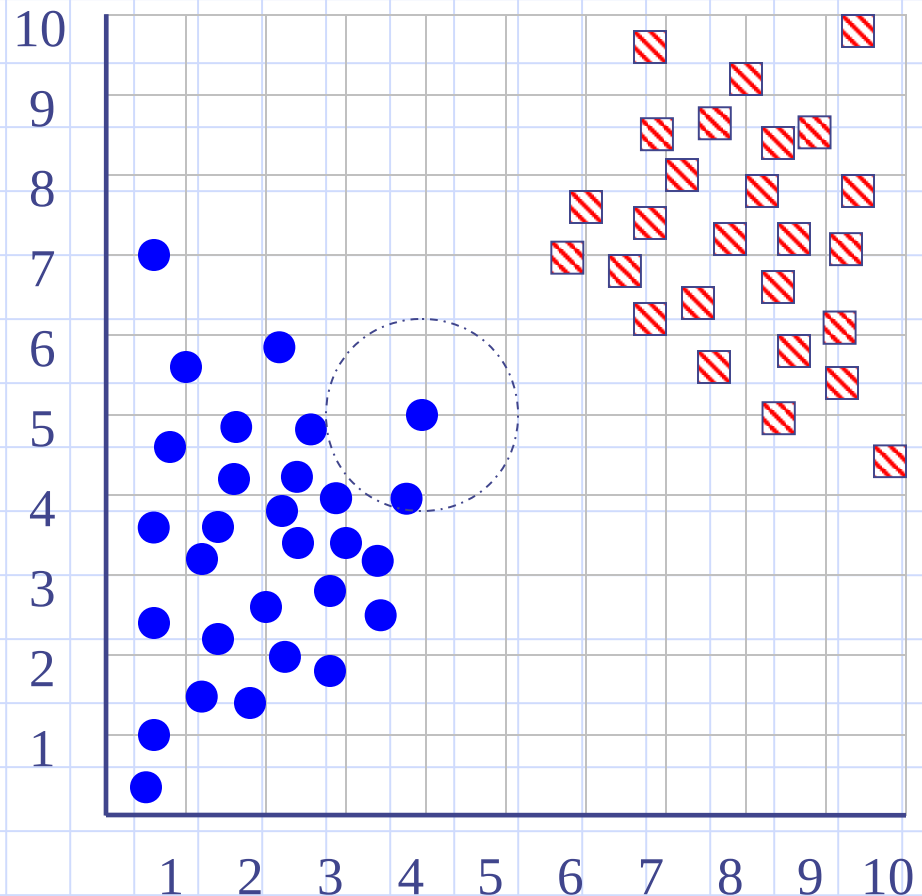
K-Vizinhos mais Próximos

- ◆ Inicialmente, os exemplos de treinamento são armazenados.
- ◆ Quando uma novo exemplo precisa ser classificado, é verificada a sua similaridade.



K-Vizinhos mais Próximos

◆ Por fim, o novo exemplo é classificado segundo a sua proximidade com os exemplos de treinamento.



K-Vizinhos mais Próximos: Parâmetro k

- ◆ O parâmetro k do k-vizinhos mais próximos é o número de vizinhos a serem considerados na classificação.
- ◆ O parâmetro k é geralmente um inteiro pequeno e ímpar (1,3,5,7,9) para evitar empates (número par de classes).
- ◆ Portanto, o 3-vizinhos mais próximos utiliza na classificação os 3 exemplos mais próximos do novo exemplo.

K-Vizinhos mais Próximos: Parâmetro k

- ◆ Quando $k = 1$ (1-vizinho mais próximo) apenas o exemplo mais próximo ao exemplo a ser classificado é considerado.
- ◆ O uso de $k = 1$ levar à classificações incorretas caso existam exemplos com ruído no conjunto de treinamento.

K-Vizinhos mais Próximos: Distância

- ◆ K-vizinhos mais próximos assume que todos os exemplos correspondem a pontos no espaço n -dimensional $\mathcal{R}^n$.
- ◆ Os vizinhos mais próximos de um exemplos são definidos por uma medida de distância, tipicamente a distância euclidiana.

K-Vizinhos mais Próximos: Distância

Table 2: Data set in attribute-value form.

	A_1	A_2	$\dots$	A_M	Y
E_1	x_{11}	x_{12}	$\dots$	x_{1M}	y_1
E_2	x_{21}	x_{22}	$\dots$	x_{2M}	y_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
E_N	x_{N1}	x_{N2}	$\dots$	x_{NM}	y_N

K-Vizinhos mais Próximos: Distância

- ◆ A distância euclidiana entre dois exemplos E_i e E_j é definida por

$$d(E_i, E_j) = \sqrt{\sum_{r=1}^M (x_{ir} - x_{jr})^2}$$

- ◆ O k-vizinhos mais próximos pode ser utilizado tanto com um atributo classe (Y) contínuo quando discreto.

K-Vizinhos mais Próximos: Classificação

- ◆ Para o caso na qual Y é discreto (problema de classificação), então deve-se encontrar os k exemplos mais próximos do exemplo a ser classificado.
- ◆ Dentre os k exemplos, verifica-se a classe mais freqüente. Essa classe é atribuída ao novo exemplo.

K-Vizinhos mais Próximos: Classificação

◆ Algoritmo de treinamento:

- Armazenar todos os exemplos de treinamento em um conjunto CT .

◆ Algoritmo de classificação:

- Dado um novo exemplo E_{novo} ;
- Sejam $E'_1, \dots, E'_k$ os k exemplos em CT mais próximos à E_{novo} .
- Retornar a classe que ocorre com maior frequência nos exemplos $E'_1, \dots, E'_k$.

K-Vizinhos mais Próximos: Regressão

- ◆ O k-vizinhos mais próximos pode ser adaptado para aproximar um atributo Y com valores contínuos.
- ◆ Para isso, basta retornar a média dos valores do variável Y para os k-vizinhos mais próximos.
- ◆ Por exemplo, pense em uma aplicação que precisa fornecer o salário de um funcionário dada a sua titulação, experiência, cargo, etc.

K-Vizinhos mais Próximos: Exemplo 1

- ◆ Calcule a distância entre o exemplo Novo e os demais:

Nr.Ex.	At.1	At.2	At.3	Classe
1	5	1	90	+
2	2	3	105	-
3	1	4	120	+
Novo	3	3	100	?

K-Vizinhos mais Próximos: Normalização

- ◆ Segundo a distância euclidiana, a distância entre os exemplos 1 e Novo é:

$$\begin{aligned}d(1, \text{Novo}) &= \sqrt{(5-3)^2 + (1-3)^2 + (90-100)^2} \\ &= \sqrt{2^2 + (-2)^2 + (-10)^2} = \sqrt{4+4+100} \simeq 10.39\end{aligned}$$

- ◆ Para resultado final, 10.39, todos os atributos contribuíram igualmente?

K-Vizinhos mais Próximos: Normalização

- ◆ Alguns atributos assumem uma faixa de valores mais ampla do que outro.
- ◆ Para evitar que esses atributos tenham um influência maior, deve ser realizada uma normalização.
- ◆ A normalização faz com que todos os atributos fiquem na mesma faixa de valores.

K-Vizinhos mais Próximos: Normalização

- ◆ Existem diversas formas de normalização, uma das mais utilizadas é a normalização linear para o intervalo [0,1]:

$$v_n = \frac{v_i - \min}{\max - \min}$$

- ◆ Na qual:

- v_n é o valor normalizado;
- v_i é o valor não normalizado;
- $\min$ e $\max$ são os valores mínimo e máximo do atributo.

K-Vizinhos mais Próximos: Exemplo 2

- ◆ Normalize os atributos da tabela do exemplo 1 segundo a normalização linear:

Nr.Ex.	At.1	At.2	At.3	Classe
1	5	1	90	+
2	2	3	105	-
3	1	4	120	+
Novo	3	3	100	?

K-Vizinhos mais Próximos: Exemplo 3

- ◆ Calcule a distância entre o exemplo Novo e o exemplo 1, dada a tabela normalizada:

Nr.Ex.	At.1	At.2	At.3	Classe
1	1	0	0	+
2	0.25	0.66	0.5	-
3	0	1	1	+
Novo	0.5	0.66	0.33	?

K-Vizinhos mais Próximos: Atributos Discretos

- ◆ Um segundo problema com a distância euclidiana é o cálculo com atributos discretos.

Nr.Ex.	At.1	At.2	At.3	Classe
1	azul	novo	ford	+
2	verde	novo	gm	-
3	verde	antigo	volks	+
Novo	azul	antigo	gm	?

K-Vizinhos mais Próximos: Atributos Discretos

- ◆ Uma forma simples de solucionar esse problema é utilizar a medida *overlap*. Nessa medida, a distância é zero se os valores são iguais ou 1 se são diferentes.
- ◆ Uma forma mais sofisticada é a *Value Difference Metric (VDM)* (Stanfil, 1986).

Value Difference Metric (VDM)

- ◆ A idéia é que valores simbólicos são similares se eles possuem correlações similares com a classe.
- ◆ Por exemplo, para um atributo “cor” que assume os valores “vermelho”, “verde” e “azul”, e para uma aplicação de identificar se um objeto é maçã ou não, os valores “vermelho” e “verde” serão considerados mais próximos.

Value Difference Metric (VDM)

$$VDM(x_{ir}, x_{jr}) = \sum_{l=1}^{N_{cl}} \left| \frac{N_{x_{ir}, C_l}}{N_{x_{ir}}} - \frac{N_{x_{jr}, C_l}}{N_{x_{jr}}} \right|^c$$

Na qual:

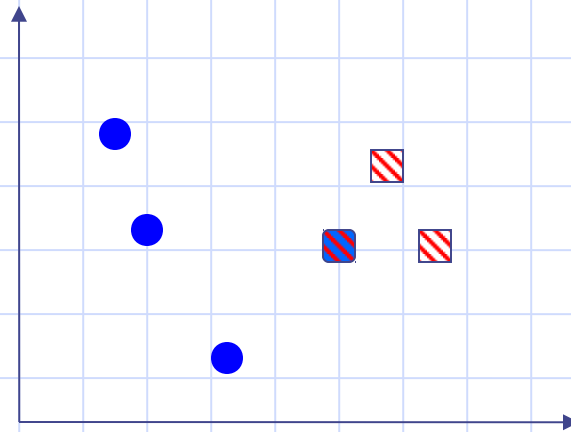
- $N_{x_{ir}}$: número de exemplos com valor x_{ir}
- N_{x_{ir}, C_l} : número exemplos com valor x_{ir} e pertencentes a classe C_l
- N_{cl} : número de classes
- c : uma constante (tipicamente 1 ou 2)

Value Difference Metric (VDM)

- ◆ VDM é uma métrica pois possui as propriedades de não negatividade, simetria e desigualdade triangular.
- ◆ (Wilson & Martinez, 2000) fizeram um amplo estudo sobre como compor medidas de distâncias heterogêneas como a distância Euclidiana e VDM e *Overlap*.

K-Vizinhos mais Próximos: Pesos

- ◆ Uma variação popular algoritmo k-vizinhos mais próximos é utilizar um peso para cada vizinho proporcional à sua distância.



K-Vizinhos mais Próximos: Pesos

- ◆ Uma sugestão é ajustar o peso do voto de cada vizinho pela equação:

$$w_i = \frac{1}{d(E_{\text{novo}}, E_i)^2}$$

- ◆ Dessa forma, quanto mais distante estiver o vizinho mais próximo (E_i) do exemplo a ser classificado E_{novo} , menor será o seu peso.

K-Vizinhos mais Próximos: Pesos

- ◆ Quando se utiliza os pesos para decidir a classe, pode-se deixar de usar apenas os k vizinhos mais próximos, e passar a utilizar todo o conjunto de treinamento.
- ◆ Isso porque exemplos muito distantes terão pouca influência na classificação do novo exemplo.

K-Vizinhos mais Próximos: Global e Local

- ◆ Se todos os exemplos são utilizados para classificar um novo exemplo, então o método é chamado de *global*.
- ◆ Se somente os vizinhos mais próximos são considerados, então o método é chamado de *local*.
- ◆ Um método global utilizado para regressão é chamado de *método de Shepard* (Shepard, 1968).

K-Vizinhos mais Próximos: Observações

◆ Algumas observações:

- K-vizinhos mais próximos é um método bastante simples, mas que provê bons resultados na prática;
- Ele é robusto a ruído e bastante efetivo quando o conjunto de treinamento não é muito pequeno;
- A melhor explicação que o método pode prover é mostrar ao usuário os vizinhos mais próximos do novo exemplo quando o método é local;

K-Vizinhos mais Próximos: Observações

◆ Algumas observações:

- Portanto, o grau de explicação desse método pode ser considerada inferior ao dos métodos simbólicos;
- O k-vizinhos mais próximos considera todos os atributos ao classificar um novo exemplo. Isso pode ser um sério problema quando existem muitos atributos irrelevantes;

K-Vizinhos mais Próximos: Observações

◆ Algumas observações:

- Para solucionar o problema de atributos irrelevantes pode-se utilizar pesos para os atributos na medida de distância;
- Outro problema é o desempenho (em tempo de execução) para classificar novos casos quando o conjunto de treinamento é muito grande;

K-Vizinhos mais Próximos: Observações

n : número de exemplos

m : número de atributos

k: número de vizinhos

Complexidade de Treino: $O(1)$

Complexidade de Teste: $O(nk+mn)$

$O(m.n)$ para calcular a distância de todos os exemplos

$O(n.k)$ para encontrar os vizinhos mais próximos

Quando o conjunto de treinamento é muito grande, torna-se computacionalmente caro encontrar os vizinhos mais próximos.

K-Vizinhos mais Próximos: Observações

n : número de exemplos

m : número de atributos

k: número de vizinhos

Complexidade de Treino: $O(1)$

Complexidade de Teste: $O(nk+mn)$

$O(m.n)$ para calcular a distância de todos os exemplos

$O(n.k)$ para encontrar os vizinhos mais próximos

Quando o conjunto de treinamento é muito grande, torna-se computacionalmente caro encontrar os vizinhos mais próximos.

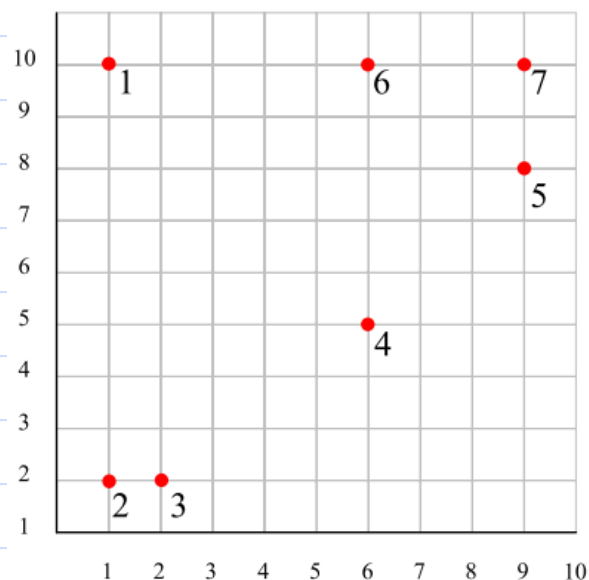
K-Vizinhos mais Próximos: Observações

- ◆ Esse problema pode ser reduzido se forem armazenados apenas exemplos prototípicos ou quando se utiliza algum método de indexação como kd-trees (Bentley, 1975), m-trees (Ciaccia, 1997) ou slim-trees (Train Jr., 2000).

Indexação

- ◆ Uma forma de indexar exemplos foi proposta por (Orchard, 1991);
- ◆ O método de Orchard é bastante simples, apesar de ser pouco utilizado por requerer $O(n^2)$ para espaço.
- ◆ Entretanto, (Ye et al., 2009) transformou esse algoritmo para ser da classe *anyspace*.

Indexação



Dataset A

	X	Y
a_1	1	10
a_2	1	2
a_3	2	2
a_4	6	5
a_5	9	8
a_6	6	10
a_7	9	10

Item	1 st NN {dist}	2 nd NN {dist}	3 rd NN {dist}	4 th NN {dist}	5 th NN {dist}	6 th NN {dist}
a_1	6 {5.0}	4 {7.1}	2 {8.0}	7 {8.0}	3 {8.1}	5 {8.2}
a_2	3 {1.0}	4 {5.8}	1 {8.0}	6 {9.4}	5 {10.0}	7 {11.3}
a_3	2 {1.0}	4 {5.0}	1 {8.1}	6 {8.9}	5 {9.2}	7 {10.6}
a_4	5 {4.2}	3 {5.0}	6 {5.0}	2 {5.8}	7 {5.8}	1 {7.1}
a_5	7 {2.0}	6 {3.6}	4 {4.2}	1 {8.2}	3 {9.2}	2 {10.0}
a_6	7 {3.0}	5 {3.6}	1 {5.0}	4 {5.0}	3 {8.9}	2 {9.4}
a_7	5 {2.0}	6 {3.0}	4 {5.8}	1 {8.0}	3 {10.6}	2 {11.3}

(Ye et al., 2009)

Indexação

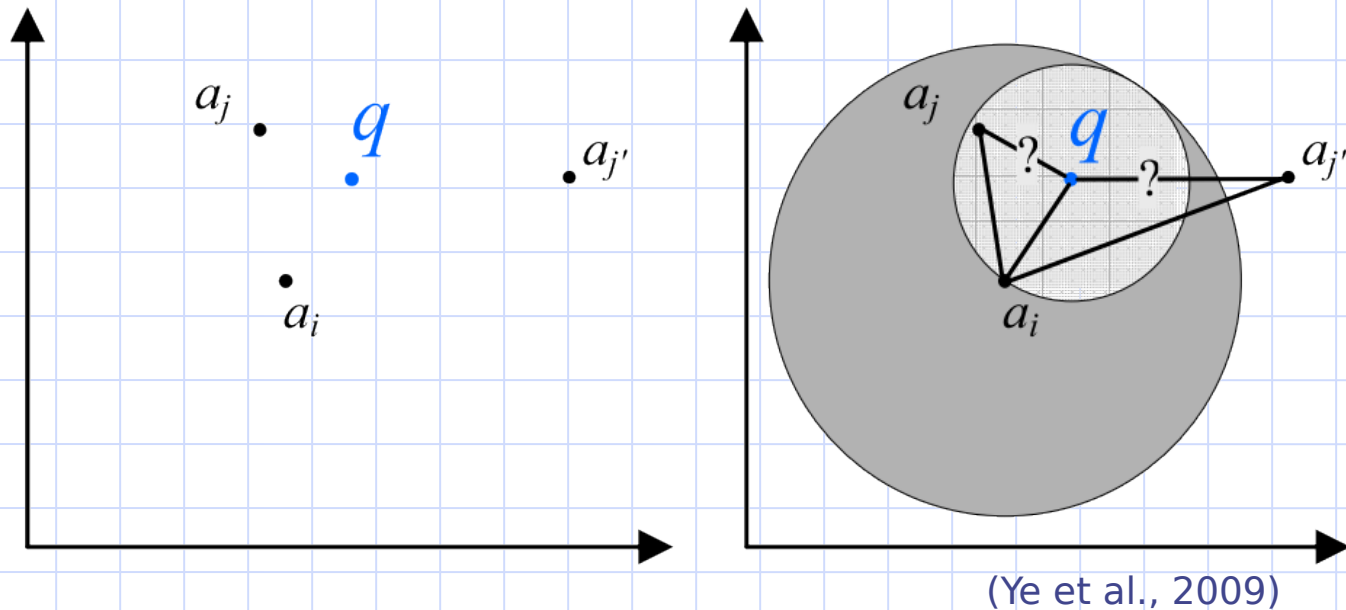
- ◆ Dado um objeto de consulta q , e a sua distância calculada para um objeto a_i ($d(a_i, q)$), qualquer objeto a_j que

$$d(a_i, a_j) \geq 2 \times d(a_i, q)$$

pode ser eliminado da busca por similaridade.

- ◆ Esse conceito é ilustrado na figura a seguir...

Indexação



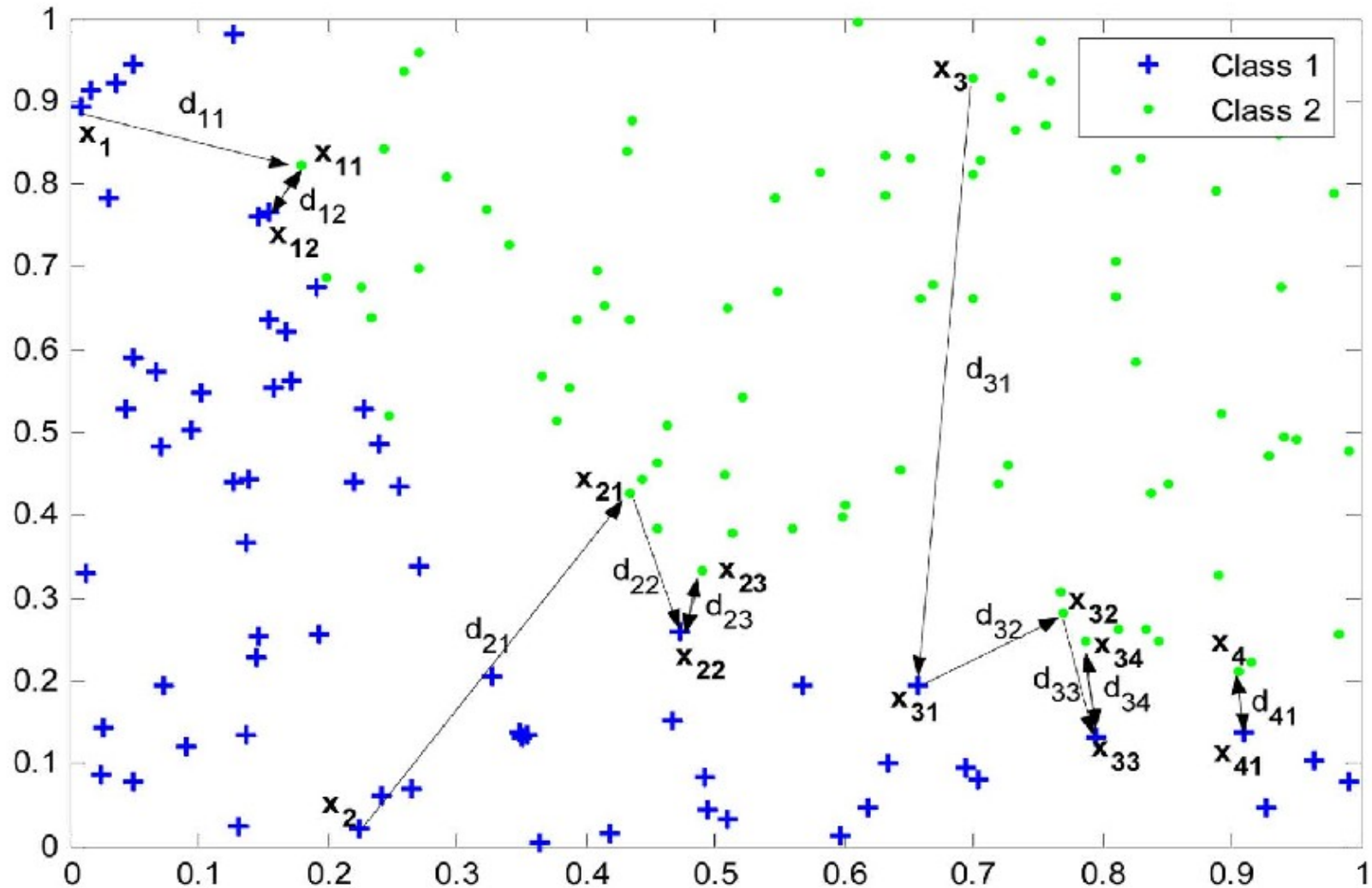
- ◆ Sabendo $d(a_i, q)$, podemos eliminar o objeto $a_{j'}$, mas não podemos eliminar a_j

Indexação

```
Function [nn.loc, nn.dist] = Orchards(P[A], q )
nn.loc = random_interger_in_range_of(1, |A|)
nn.dist = dist(ann.loc, q)
index = 1
While P[ann.loc].dist[index] < 2 * nn.dist AND index < |A| do
  node = P[ann.loc].node[index]
  If node is not yet tested then
    d = dist(anode, q)
    If d < nn.dist then
      nn.dist = d
      nn.loc = node
      index = 1
    Else
      index = index + 1
    EndIf
  Else
    index = index + 1
  EndIf
EndWhile
```

(Ye et al., 2009)

Template Reduction for KNN (TRKNN)



Template Reduction for KNN (TRKNN)

Movimento	1-NN	TR-1NN	3-NN	TR-3NN	5-NN	TR-5NN
Baixo	100% (0,0)	95% (10,80)	97% (9,48)	97% (9,48)	97% (9,48)	97% (9,48)
Cima	100% (0,0)	100% (0,0)	100% (0,0)	99% (3,16)	100% (0,0)	99% (3,16)
Círculo	97% (9,48)	99% (3,16)	97% (6,74)	98% (6,32)	96% (8,43)	95% (8,49)
Diagonal Inferior Esquerda	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)
Diagonal Inferior Direita	92% (17,51)	89% (17,91)	91% (20,24)	83% (29,07)	90% (21,60)	78% (30,84)
Diagonal Superior Direita	97% (9,48)	96% (12,64)	97% (9,48)	96% (12,64)	97% (9,48)	94% (15,77)
Diagonal Superior Esquerda	100% (0,0)	99% (3,16)	100% (0,0)	99% (3,16)	100% (0,0)	95% (15,81)
Direita	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)
Esquerda	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)
Quadrado	96% (12,64)	96% (0,0)	96% (12,64)	96% (12,64)	100% (0,0)	96% (12,64)
Tennis	97% (4,83)	91% (19,11)	94% (9,66)	87% (27,90)	92% (15,49)	86% (27,96)
Z	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)	100% (0,0)
Média	98.16% (2,47)	97,08% (2,49)	97.66% (2,96)	96,24% (3,66)	97,33% (3,32)	94,99% (4,04)

Tabela 1

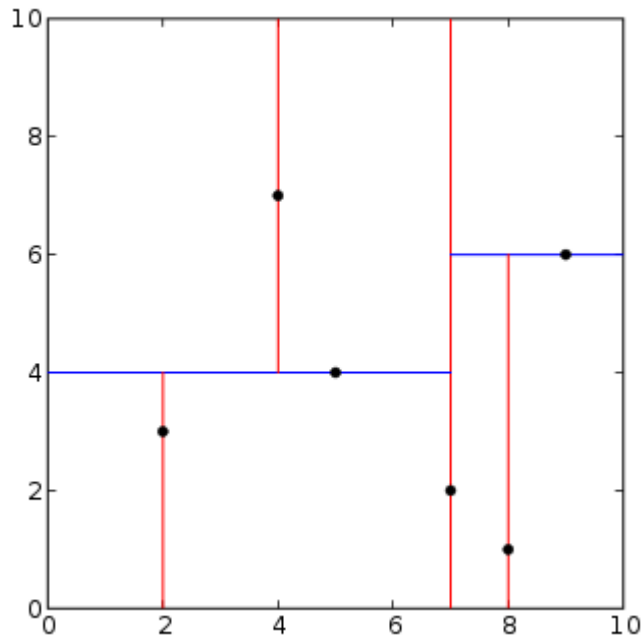
Taxa de acerto obtida utilizando validação cruzada de 10 partições para cada tipo de movimento utilizando TRKNN e KNN

Template Reduction for KNN (TRKNN)

Tempo médio de execução para
3600 movimentos

Tempo(KNN) = 90,33s (3,15)

Tempo(TRKNN) = 26,45s (2,51)



KD-Tree

X

(7,2)

Y

(5,4)

(9,6)

X

(2,3)

(4,7)

(8,1)

Índice Invertido

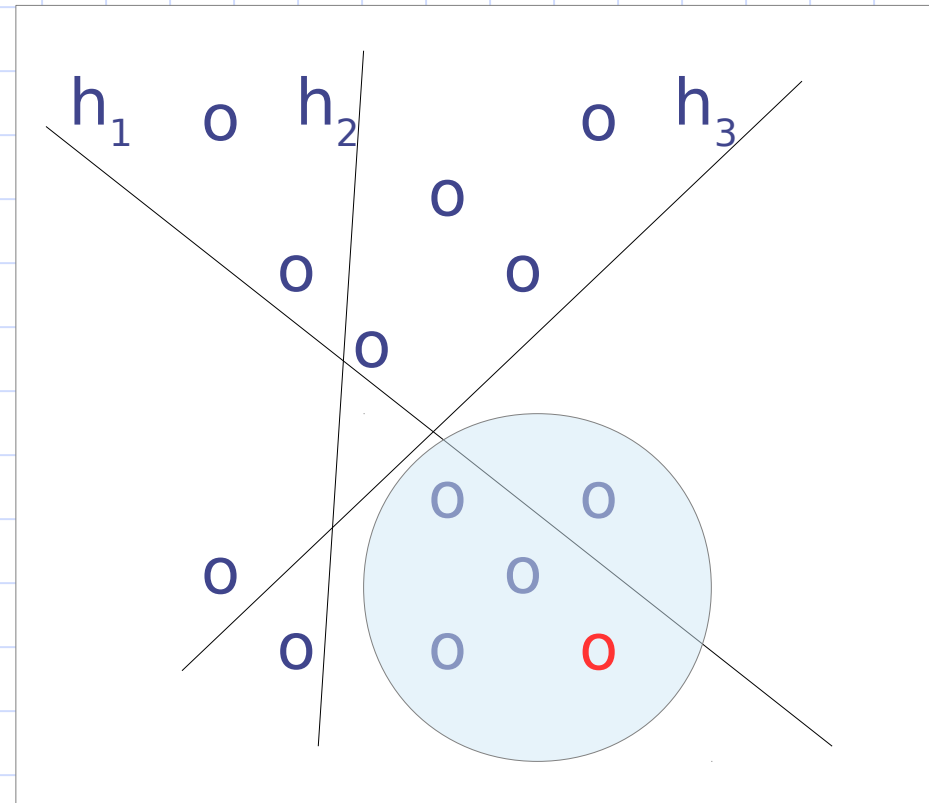
D1: “promoção iphone” spam
D2: “promoção samsung” spam
D3: “projeto ia” não-spam
D4: “filtro spam” não-spam
Novo exemplo: “projeto filtro”

palavra	documento
promoção	D1, D2
iphone	D1
samsung	D2
projeto	D3
filtro	D4
ia	D3
spam	D4

Locality-Sensitive Hashing (LSH)

Hiper-planos aleatórios $h_1 \dots h_k$

Comparar somente
com os vizinhos que
estiverem na
mesma partição



Improving Instance Selection via Metric Learning

Eduardo Zrate Max

Computer Faculty

UFMS

Brazil

eduardo.max@gmail.com

Ricardo Marcacine

Computer Faculty

UFMS

Brazil

ricardo.marcacine@ufms.br

Edson Takashi Matsubara

Computer Faculty

UFMS

Brazil

edsontm@facom.ufms.br

Abstract—The k -Nearest Neighbor (k -NN) rule is widely used for classification tasks because of its simplicity and efficiency. However, a well-known drawback of k -NN is its dependence on the quality of the training set, since the k -NN makes no assumption about the importance of each instance. In fact, the existence of noisy and superfluous instances in the training set tends to increase the classification error rate. Thus, instance selection methods are useful to identify which instances belonging to the training set will be considered in the k -NN classifier. Our proposal shows a simple and effective way to improve instance selection methods using metric learning. The idea of our proposal relies on a pure geometric intuition that metric learning transforms the input space where points in the same class are simultaneously near each other and far from points in the other classes. In a more “organised” space, we show that instance selection methods can benefit from this transformed space. We carried out an experimental evaluation to compare the instance selection with and without metric learning on UCI benchmark data sets. The results reveals that the combination of metric and instance selection is very welcome. All tested instance selection methods improved significantly.

Index Terms—metric learning, instance selection

I. INTRODUCTION

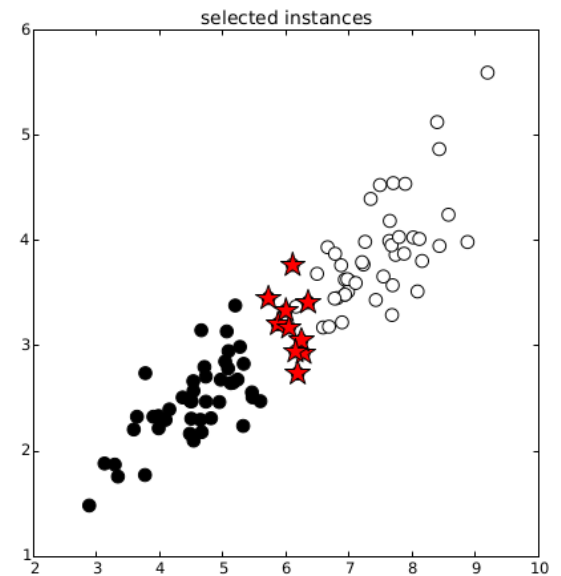
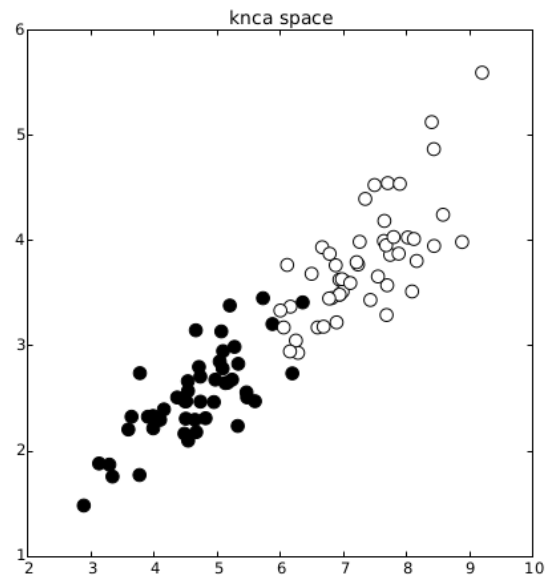
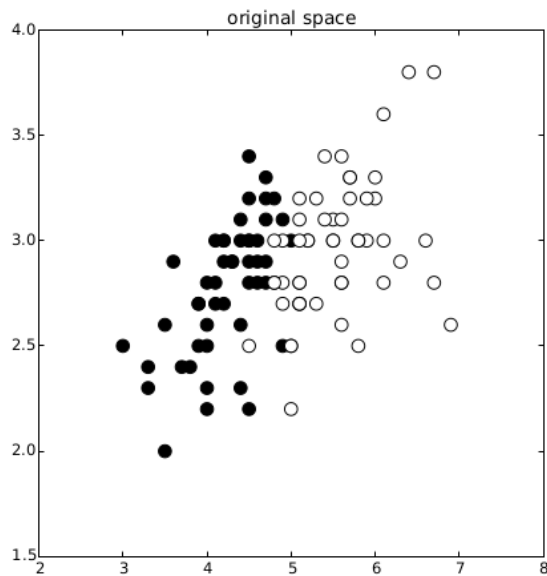
The k -Nearest Neighbor Classifier (k -NN) is a well-known supervised classification algorithm based on a simple and intuitive rule: the class of a new instance is the majority class of their most k similar instances [1]. The popularity of the k -NN is also justified because of its strong statistical basis: “in the large sample case, this simple rule has a probability of error which is less than twice the Bayes probability of

Although there is a wide variety of instance selection methods, the performance of the instance selection depends on the distance measure used to identify the nearest neighbor instances [5]–[7]. In fact, the choice of the distance measure is a non-trivial task because it depends on the (i) data type, (ii) the importance of some attributes, and (iii) the application domain [8]. This kind of information is generally not available, thereby promoting the widespread use of traditional measures such as Euclidean distance without a study of their suitability for the problem. Thus, we assume that the learning of an appropriate distance measure for a given dataset can significantly improve instance selection task – and consequently reduce the error rate of the k -NN classifier by considering only representative instances in the training set.

Considering the advantages of using an appropriate distance measure for nearest neighbor-based classification problems, in this paper we experimentally show that the use of metric learning before an instance selection method improves, in most cases, the accuracy of k -NN.

To illustrate our proposal, Figure 1 shows an example of using metric learning in instance selection process. We select the Petal length and Sepal width attributes of the well-known Iris UCI dataset, and all the instances of the classes Versicolor and Virginica. In the left frame, the instances are plotted from original euclidean space, where the colors identify the two different classes. We learn an appropriate metric for the dataset by using the problem formulation defined in Equation 4. In the middle frame the instances are plotted considering this new metric space. The inter-class and intra-class separation

Transformação do espaço



Resultados

instsel	#ds	Without KNCA		With KNCA	
		Avg $\pm$ StdDev	#Sel	Avg $\pm$ StdDev	#Sel
CHC	1	0.904 $\pm$ 0.058	316.5	0.980 $\pm$ 0.023	297.3
CHC	2	0.789 $\pm$ 0.051	87.0	0.804 $\pm$ 0.033	93.0
CHC	3	0.835 $\pm$ 0.021	113.6	0.850 $\pm$ 0.021	111.3
CHC	4	0.897 $\pm$ 0.050	180.2	0.930 $\pm$ 0.045	176.7
CHC	5	1.000 $\pm$ 0.000	44.6	1.000 $\pm$ 0.000	52.6
CHC	6	0.495 $\pm$ 0.057	85.3	0.531 $\pm$ 0.071	82.3
CHC	7	0.789 $\pm$ 0.035	106.0	0.697 $\pm$ 0.026	97.0
CHC	8	0.909 $\pm$ 0.105	452.4	0.945 $\pm$ 0.082	445.3
CNN	1	0.833 $\pm$ 0.100	112.2	0.977 $\pm$ 0.020	49.5
CNN	2	0.659 $\pm$ 0.056	48.0	0.731 $\pm$ 0.075	47.6
CNN	3	0.737 $\pm$ 0.031	58.6	0.818 $\pm$ 0.024	59.3
CNN	4	0.860 $\pm$ 0.073	53.8	0.834 $\pm$ 0.061	48.3
CNN	5	0.500 $\pm$ 0.000	2.3	0.500 $\pm$ 0.000	2.0
CNN	6	0.490 $\pm$ 0.030	14.3	0.531 $\pm$ 0.097	15.6
CNN	7	0.625 $\pm$ 0.071	53.6	0.760 $\pm$ 0.059	32.6
CNN	8	0.877 $\pm$ 0.054	34.5	0.885 $\pm$ 0.064	22.9
ENN	1	0.909 $\pm$ 0.058	523.6	0.975 $\pm$ 0.032	547.6
ENN	2	0.724 $\pm$ 0.091	113.0	0.705 $\pm$ 0.033	109.0
ENN	3	0.825 $\pm$ 0.021	145.0	0.841 $\pm$ 0.012	149.3
ENN	4	0.826 $\pm$ 0.077	265.2	0.856 $\pm$ 0.055	280.7
ENN	5	1.000 $\pm$ 0.000	100.0	1.000 $\pm$ 0.000	100.0
ENN	6	0.472 $\pm$ 0.072	119.0	0.521 $\pm$ 0.016	126.6
ENN	7	0.760 $\pm$ 0.025	146.0	0.736 $\pm$ 0.027	160.6
ENN	8	0.897 $\pm$ 0.086	886.7	0.936 $\pm$ 0.096	890.0
RMHC	1	0.936 $\pm$ 0.055	56.0	0.978 $\pm$ 0.022	56.0
RMHC	2	0.614 $\pm$ 0.062	14.0	0.611 $\pm$ 0.056	14.0
RMHC	3	0.839 $\pm$ 0.029	17.0	0.842 $\pm$ 0.039	17.0
RMHC	4	0.842 $\pm$ 0.100	31.0	0.882 $\pm$ 0.049	31.0
RMHC	5	1.000 $\pm$ 0.000	9.3	1.000 $\pm$ 0.000	9.3
RMHC	6	0.469 $\pm$ 0.154	13.0	0.548 $\pm$ 0.054	13.0
RMHC	7	0.746 $\pm$ 0.041	17.0	0.734 $\pm$ 0.092	17.0
RMHC	8	0.905 $\pm$ 0.082	89.0	0.930 $\pm$ 0.097	89.0
average		0.780 $\pm$ 0.160	134.61	0.808 $\pm$ 0.159	132.60

Referências

◆ Referências:

- Bentley, J. *Multidimensional binary search trees used for associative searching*. Communications of the ACM, 18(9), 509-517, 1975.
- Ciaccia, P.; Patella, M.; Zezula, P. *M-tree: an efficient access method for similarity search in metric spaces*. Proceedings of VLDB, 426-435, 1997.
- Mitchell, T. *Machine Learning*. McGraw-Hill, 1997.
- Orchard, M. T. *A fast nearest-neighbor search algorithm*. International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 2297-2300, IEEE Computer Society Press, 1991.
- Datar, Mayur, et al. "Locality-sensitive hashing scheme based on p-stable distributions." Proceedings of the twentieth annual symposium on Computational geometry. ACM, 2004.

Referências

◆ Referências:

- Traina, Jr., C.; Traina, A.; Seeger, B.; Faloutsos, C. *Slim-trees: high performance metric trees minimizing overlap between nodes*. Proceedings of ETBT, 51-65, 2000.
- Shepard, D. *A two-dimensional interpolation function for irregularly spaced data*. Proceedings of the 23rd National Conference of the ACM, 517-523, 1968.
- Stanfil, C.; Waltz, D. *Towards Memory-based reasoning*. Communications of the ACM, 29, 1213-1228, 1986.

Referências

◆ Referências:

- Ye, L.; Wang, X.; Keogh, E.; Mafra-Neto, A. *Autocannibalistic and Anyspace Indexing Algorithms with Applications to Sensor Data Mining*. SIAM International Conference on Data Mining, 2009.
- Wilson, D. R.; Martinez, T. R. Improved heterogeneous distance functions. *Journal of Artificial Intelligence Research (JAIR)*, 6:1–34, 1997.